

jaato TUI — User Manual

Generated by the jaato-tui-driven-tests harness

2026-05-08

Contents

Introduction	1
Getting started — running the TUI	2
First run — auto-start	2
Reconnecting to an existing session	2
Connecting to a specific socket	2
Running the daemon by hand	2
Checking + stopping the daemon	3
Selecting a profile or agent at startup	3
A non-interactive one-shot	3
Ctrl+B — Budget panel (token usage by category)	4
Exit command — Detach / end-session menu	5
Initial TUI state — banner + prompt	6

Introduction

This manual documents the jaato TUI client (the `jaato` command) — the terminal interface for interacting with the jaato daemon. Sections are organized into:

- **Reference catalogs** (auto-generated): keybindings, slash commands, tools, profile knobs. These reflect the live framework state on the day this manual was built.
- **Feature walkthroughs** (auto-generated capture + operator-edited caption): one section per feature, showing what you see and what to do with it.
- **Appendix**: tear-down, troubleshooting, where to read the source.

This document is built by a self-documenting harness — the harness drives a real TUI session via `tmux` and captures each feature’s screen state directly. The captures are real terminal output, not synthesized mockups.

If something in here looks wrong or stale, regenerate the manual:

```
cd jaato-tui-driven-tests
python -m harness all
```

The walk phase auto-starts a daemon on `/tmp/jaato.sock` if one isn’t already running — same default behaviour as launching `jaato` directly.

Getting started — running the TUI

The TUI is launched with the `jaato` command. The daemon (the “server”) is launched with `jaato-server`. Both come from the `jaato-tui` and `jaato-server` packages respectively; once the project’s virtualenv is activated, both commands are on `PATH` — there is no need to invoke `python -m server` or `python rich_client.py` manually.

First run — auto-start

The simplest way to start a session is:

```
$ jaato
```

If no daemon is listening on the default IPC socket (`/tmp/jaato.sock`), the TUI **spawns one in the background** and connects to it. The default config directory is `.jaato/` in the current working directory; if that directory does not exist, the TUI will exit and ask you to scaffold it first:

```
$ jaato --init
```

`--init` writes a starter `.jaato/` next to the current directory with example agents, profiles, references, and an `.env` template. Edit those files, then re-run `jaato`.

Reconnecting to an existing session

Re-running `jaato` on a workspace where the daemon is already listening on `/tmp/jaato.sock` will **resume the default session** — the same conversation history, todos, plan, and budget you had in the previous TUI window. This is the expected behaviour when you close the TUI (Ctrl+D) and come back later: the daemon stays alive in the background and your context is waiting for you.

If you want to start fresh without losing the prior session, pass `--new-session`:

```
$ jaato --new-session
```

The default session is left intact on the daemon — you can switch back to it later from the TUI’s session-management commands.

Connecting to a specific socket

The default IPC socket path is `/tmp/jaato.sock`. If you run the daemon on a different socket (for example to isolate workspaces or to run multiple daemons side-by-side), point the TUI at it explicitly:

```
$ jaato --connect /tmp/scratch.sock
```

When `--connect` is set, auto-start is disabled by default for any non-default path — the TUI assumes you already have the daemon running where you want it. Use `--no-auto-start` to opt out of auto-start unconditionally.

Running the daemon by hand

For most users, auto-start is sufficient. The cases where you want to run the daemon manually are:

- You want the daemon to outlive the TUI window (so reconnects are fast, even after a system reboot of the TUI’s terminal multiplexer process).
- You want to expose the daemon over WebSocket so a remote client can connect.
- You want to inspect the daemon’s logs in real time (`/tmp/jaato.log` rotates by default; the foreground daemon prints directly to its terminal).

Run the daemon manually with:

```
$ jaato-server --ipc-socket /tmp/jaato.sock --daemon
```

To also expose a WebSocket endpoint:

```
$ jaato-server --ipc-socket /tmp/jaato.sock --web-socket :8080 --daemon
```

The `--daemon` flag detaches the server from the current terminal. Without it the daemon runs in the foreground (useful for tailing logs).

Checking + stopping the daemon

```
$ jaato-server --status      # show pid, sockets, uptime
$ jaato-server --stop        # send graceful shutdown
```

Selecting a profile or agent at startup

Sessions can be created with a runtime **profile** (model + plugins + GC) or an **agent** (parameterized prompt + persona). Both are read from `.jaato/profiles/` and `.jaato/agents/` respectively. At startup, pass either or both:

```
$ jaato --profile researcher --agent code-explainer
```

`--new-session` is implied when `--profile` or `--agent` is set — profiles and agents only apply to fresh sessions. Existing sessions keep the profile they were created with.

A non-interactive one-shot

For scripting or piping output into another tool, ask the TUI for a single response and exit:

```
$ jaato --prompt "What changed in the latest commit?"
```

The TUI runs the prompt against the default session, prints the response, and exits. Auto-start still applies — this is a useful pattern for CI hooks and editor integrations that don't want a persistent terminal.

If you want to seed the session with an opening prompt but **stay interactive** afterwards, use `--initial-prompt (-i)` instead.

Ctrl+B — Budget panel (token usage by category)

View real-time token consumption across system, plugin, enrichment, conversation, and thinking categories.

The budget panel displays a live breakdown of token usage by source category. This is useful for understanding where your context budget is being consumed during a session. Each category shows the number of tokens used, a garbage-collection (GC) status indicator, and a visual bar representing usage as a proportion of the total.

The panel is organized as a hierarchical list. The top-level row shows the total token count for the entire session. Below that, each source category (System, Plugin, Enrichment, Conversation, Thinking) is listed with its own token count and status. You can navigate between categories using the arrow keys and drill down to see more details about locked or partial categories.

On screen:

Token Usage (Total: 29.8K tokens)			
Source	Tokens	GC	Usage
▶ System	25.4K	🔒	
Plugin	4.4K	🔒	
Enrichment	0	○	
Conversation	0	◐	
Thinking	0	○	

Notes

- The **GC column** shows the garbage-collection status: 🔒 (locked — cannot be garbage-collected), ◐ (partial — partially eligible for GC), ○ (ephemeral — eligible for GC)
- The **Usage bar** is a visual representation of each category's token consumption relative to the total; filled blocks (■) represent used tokens, empty blocks (░) represent available capacity
- The ▶ **symbol** next to System indicates that category can be expanded to show sub-items; press Enter to drill down
- Total token count is displayed in the panel title and updated in real time as the session progresses
- Navigation: use ↑ ↓ to move between categories, Enter to drill down into expandable items, Esc to close or return to the previous level

Related sections

- [initial-banner](#)

Exit command — Detach / end-session menu

Choose how to end your session: detach to reconnect later, or end it permanently.

When you type `exit` at the prompt, the TUI presents a menu with three options for managing your session. You can detach from the session (leaving it running on the server so you can reconnect later), end the session entirely (deleting it from the server), or return to the session without exiting.

On screen:

```
Exit options:
[d] Detach (keep session, can reconnect later)
[e] End session (delete session from server)
[r] Return to session
```

Choice [d/e/r]:

Notes

- Press `d` to detach: the session remains active on the server and can be reconnected to later
- Press `e` to end the session: the session is deleted from the server and cannot be recovered
- Press `r` to return: cancels the exit and returns to the active session
- The prompt waits for a single keystroke; no Enter key needed

Related sections

- [initial-banner](#)

Initial TUI state — banner + prompt

The TUI displays a header banner with session metadata, model information, and context budget, followed by instructional text and a ready prompt.

When you start the Jaato TUI, the first screen shows a structured header containing key session information: your session ID, workspace path, toolblock status, and permission mode. Below that, the model provider and context budget are displayed, giving you immediate visibility into your current AI configuration and token availability. The main content area then presents helpful guidance text explaining the core capabilities—file editing, tab completion, available commands, and the connected AI model—before settling into the user prompt where you can begin entering commands.

On screen:

```
Session: 20260508_082133 | Workspace: ~/Sources/Jaato-framework-and-examples/
· Main Agent
Provider: openrouter | Model: anthropic/claude-haiku-4.5 | Context: 85% ava

We now can edit files from the workspace panel
Tab completion enabled. Use file for files, @path for sandbox paths,
prompt for skills.
Type 'help' for commands, Ctrl+G for editor, Ctrl+F for search.
Connected to openrouter/anthropic/claude-haiku-4.5 (OpenRouter API key
(JAATO_OPENROUTER_API_KEY))
Session created: Session 2026-05-08 08:21 (20260508_082133)
```

User>

Notes

- The top line displays the session ID and workspace path at a glance
- The second line shows the agent status (spinning indicator) and agent name
- The third line reports the AI provider, model name, and context availability as a percentage
- The main text area provides quick-reference guidance for common operations: file editing, tab completion syntax, help command, and keyboard shortcuts
- The prompt line (User>) is where you enter your first command or question